1) Find T(n) for the following algorithm:

```
Algorithm UniqueElements ( A[0 … n-1])
for i ← 0 to n-2 do
        for j ← i+1 to n-1 do
                if A[i] = A[j] return false
```

Solution

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$$

---

2) Find the complexity of the following algorithms, where A, B are arrays, n is the input size:
[Note: your answer should include steps that derive to the final answer]

i)
```
for i = 1 to n Do
    A[i] = 1 / i
    B[i] = i2
endfor
InsertionSort(A, n)
BubbleSort(B, n)
```

Assume that: <u>InsertionSort</u> and <u>BubbleSort</u> complexity is n^2

ii) i = 1

```
while i < n do
    i = 2 × i
endwhile
j =1
while j < 10 do
    j = 2 × j
endwhile
```

Solution

i) O(n) + O(n2) + O(n2) = O(n2)
ii) O(log n) + O(1) = O(log n)

---

3) Use Master Theorem to find the O notation for the following recurrence relations.
[Note: your answer should include steps that derive to the final answer]

i) T(n) = 4T(n/2) + n
ii) T(n) = T(n/2) + n

Solution

i) $a = 4, b = 2, d = 1, a > b^d \Rightarrow T(n) \in \Theta(n^{\log_2 4}) \Rightarrow \Theta(n^2)$
ii) $a = 1, b = 2, d = 1, \quad a < b^d \Rightarrow T(n) \in \Theta(n)$

---

4)

Use Master Theorem to find the O notation for the following recurrence relations.
i) $T(n) = T(n/2) + 1$ (3 marks)
ii) $T(n) = 4T(n/2) + n^2$ (3 marks)

Solution

i) Here $a = 1$, $b = 2$, $d = 0$, $a = b^d$
$T(n) \in \Theta(\log(n))$
ii) Here $a = 4$, $b = 2$, $d = 2$, $a = b^d$
$T(n) \in \Theta(n^2 \log n)$

---

5) Find O, Ω, θ for the following:
   a. $T(n) = 100n + 6$
   b. $T(n) = n^3 + n + 5$
   c. $n\sqrt{n} + n\log_2(n) + 2$

---

6) Solve this recurrence

$$T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log n.$$

---

7)

Which of the following best describes the useful criterion for comparing the efficiency of algorithms?

- A) Time
- B) Memory
- C) Both of the above
- D) None of the above

Solution: C

8)

How is time complexity measured?

- A) By counting the number of statements in an algorithm
- B) By counting the number of primitive operations performed by the algorithm on a given input size
- C) By counting the size of data input to the algorithm
- D) None of the above

Solution: B

9)

Which of the following does NOT belong to the family of notations?

- A) Big (O)
- B) Big (Ω)
- C) Big (θ)
- D) Big (⋈)

Solution: D

---

10)

What will be the time complexity for following fragment of code?

```
for(var i=0;i<n;i++)
    i*=k
```

- A) $O(n)$
- B) $O(k)$
- C) $O(log_n k)$
- D) $O(log_k n)$

Solution: D

---

11)

What is the Big O time complexity of the following?

```
for(var i=0;i<n;i++)
    for(var j=0;j<m;j++)
```

A) $O(n)$

B) $O(m)$

C) $O(nm)$

D) $O(n+m)$

Solution: C

---

12)

Algorithm **A** and **B** have a worst-case running time of $O(n)$ and $O(logn)$, respectively. Therefore, algorithm B always runs faster than the algorithm A.

A) True

B) False

Solution: B

Reason: Wrong keyword is <u>always</u>. Algorithm B is faster than Algorithm A for all points where n > n0 and not always, because always mean it is faster for any point. But literally, if n < n0, algorithm A may be faster than B.

---

13)

What is the time, space complexity of following code:

```
int a = 0, b = 0;
for (i = 0; i < N; i++) {
    a = a + rand();
}
for (j = 0; j < M; j++) {
    b = b + rand();
}
```

Options:

1. O(N * M) time, O(1) space
2. O(N + M) time, O(N + M) space
3. O(N + M) time, O(1) space
4. O(N * M) time, O(N + M) space

Solution: 3

---

14)

What is the time complexity of following code:

```
int a = 0;
for (i = 0; i < N; i++) {
    for (j = N; j > i; j--) {
        a = a + i + j;
    }
}
```

Options:

1. O(N)
2. O(N*log(N))
3. O(N * Sqrt(N))
4. O(N*N)

Solution: 4

---

15)

What is the time complexity of following code:

```
int i, j, k = 0;
for (i = n / 2; i <= n; i++) {
    for (j = 2; j <= n; j = j * 2) {
        k = k + n / 2;
    }
}
```

Options:

1. O(n)
2. O(nLogn)
3. O(n^2)
4. O(n^2Logn)

Solution: 2

---

16)

What is the time complexity of following code:

```
int a = 0, i = N;
while (i > 0) {
    a += i;
    i /= 2;
}
```

Options:

1. O(N)
2. O(Sqrt(N))
3. O(N / 2)
4. O(log N)

Solution: 4

---

17)

For the functions, n^k and c^n, what is the asymptotic relationship between these functions?

Assume that k >= 1 and c > 1 are constants.

Choose all answers that apply:

(A) n^k is $O(c^n)$

(B) n^k is $\Omega(c^n)$

(C) n^k is $\Theta(c^n)$

Solution: A

---

18)

For the functions, $\log_2 n$ and $\log_8 n$, what is the asymptotic relationship between these functions?

Choose all answers that apply:

(A) $\log_2 n$ is $O(\log_8 n)$

(B) $\log_2 n$ is $\Omega(\log_8 n)$

(C) $\log_2 n$ is $\Theta(\log_8 n)$

Solution: A, B and C (all correct)

Note that: 2 and 8 are constant. You can ignore them to be <u>log n</u> VS <u>log n</u>. So, it will be O, Ω, θ

---

19)

What is the asymptotic relationship between the functions $n^3 \log_2 n$ and $3n \log_8 n$?

Choose all answers that apply:

(A) $n^3 \log_2 n$ is $O(3n \log_8 n)$

(B) $n^3 \log_2 n$ is $\Omega(3n \log_8 n)$

(C) $n^3 \log_2 n$ is $\Theta(3n \log_8 n)$

Solution: B

---

20)

For the functions, 8^n and 4^n, what is the asymptotic relationship between these functions?

Choose all answers that apply:

(A) 8^n is $O(4^n)$

(B) 8^n is $\Omega(4^n)$

(C) 8^n is $\Theta(4^n)$

Solution: B

---

21)

For the functions, $\log_2 n^{\log_2 17}$ vs. $\log_2 17^{\log_2 n}$, what is the asymptotic relationship between these functions?

Choose all answers that apply:

(A) $\log_2 n^{\log_2 17}$ is $O(\log_2 17^{\log_2 n})$

(B) $\log_2 n^{\log_2 17}$ is $\Omega(\log_2 17^{\log_2 n})$

(C) $\log_2 n^{\log_2 17}$ is $\Theta(\log_2 17^{\log_2 n})$

Solution: A, B and C (all correct)

Reason: this part $\log_2 17^{\log_2 n}$ can be equal = $\log_2 n^{\log_2 17}$. This is rule in logarithmic where 17 and n can be swiped together. So, the two terms are equal, which mean it can be O, Ω, θ.

---

22)

Look at the following algorithms

```
int n;
int A[100];
void (x){
int i;
for (i=n/2;i > = 1; i -- )
y(i);
}
}
void  y(int i)
{
.........;
.........;
.........;
}
```

Let complexity of y is $O(\log_2 n)$. Then the complexity of x will be

- **A.** $O(n^2)$
- **B.** $O(n^2 \log_2 n)$
- **C.** $O(n \log_2 n)$
- **D.** $O(n)$

Solution: C

23)

The time complexity of the following C function is (assume n > 0)

```
int recursive (int n)
{
if (n == 1)
return(1);
else
return(recursive(n- 1) + recursive(n - 1));
}
```

- **A.** O(n)
- **B.** O(n logo)
- **C.** O (n2)
- **D.** $O(2^n)$

Solution: D

Reason: $T(n) = 1 + T(n-1) + T(n-1) = 2T(n-1) + 1$

We solve by tree method and get $T(n) = 2^1 + 2^2 + 2^3 = 2(2^n - 1)$ [this is new rule]

---

24)

The recurrence equation:

$T(1) = 1$

$T(n) = 2T(n-1) + n, n \geq 2$

Evaluates to

- **A.** $2^{n+1} - n - 2$
- **B.** $2^{n} - n$
- **C.** $2^{n+1} - 2n - 2$
- **D.** $2^{n}+n$

Solution: A

Reason: solve by tree method and sum time of each level will be equal

= n + 2n-2 + 4n-8 + 8n-24 + …… [Simplify this]

= n + 2^1(n-1) + 2^2(n-2) + 2^3(n-3) + ……

As you see, the terms construct the following pattern

$$\mathrm{T(n)} = \sum_{i=1}^{n} 2^{i}\,(\mathrm{n} - \mathrm{i})$$

This is new summation rule, it's value = 2^{n+1} -n -2

---

25)

Let T(n) be a function defined by the recurrence
T(n) - 2T(n/2) + √n for n≥2 and T(1) = 1
Which of the following statements is TRUE?

- A. $T(n) = \Theta(\log n)$
- B. $T(n) = \Theta(\sqrt{n})$
- C. $T(n) = \Theta(n)$
- D. $T(n) = \Theta(n \log n)$

Solution: C

You can solve by master method

---

26)

Suppose T(n) = 2T(n/2) + n, T(0) = T(1) = 1 Which one of the following is FALSE?

- A. $T(n) = 0(n^2)$
- B. $T(n) = \Theta(n \log n)$
- C. $T(n) - \Omega(n^2)$
- D. $T(n) = O(n\log n)$

Solution: C

Reason: you can use master method to get that time complexity here is nlogn. So, A B D are correct. C only is wrong.

27)

What is the time complexity of the following recursive function:

```
int DoSomething ( int n)
{
if {n< = 2)
return 1;
else
return DoSomething (floor (sqrt (n)))+n;
}
```

- A. $\Theta(n^2)$
- B. $\Theta(n \log_2 n)$
- C. $\Theta(\log_2 n)$
- D. $\Theta(\log_2 \log_2 n)$

Solution: D

Reason: T(n) = $T(\sqrt{n}) + 1$

Review master method video part for the same example which = log log n

28)

The running time of an algorithm T(n), where 'n' is the input size is given by
T(n) = 8T(n/2) + qn. if n> 1
p, if n = 1
where p, q are constants. The order of this algorithm is

- A. $O(n^2)$
- B. $O(n^n)$
- C. $O(n^3)$
- D. $O(n)$

Solution: C

Use master method to solve it